Minor Assignment 06 — Solutions

Game Programming with C++ (CSE 3545) • sf::View + Player & Zombie classes • Study Guide

This assignment puts together the pieces of the actual Zombie Arena game: a **2D camera (sf::View)** that follows the player, a **Player class** that owns its own texture/sprite, a **Zombie class** with three subtypes, plus the event-handling state machine that drives everything.

Three big ideas:
1. sf::View — a rectangle that defines what slice of the world the window shows. Move the view, and the whole scene appears to scroll; resize it, and you zoom.
2. Class encapsulation — Player and Zombie each own their Texture + Sprite + position internally, and expose only the operations the rest of the game needs (spawn, update, getPosition…).
3. State machine — the game is always in exactly one of PLAYING / PAUSED / GAME_OVER / LEVELING_UP. The Return / Num keys move between states.

Q1. Graphical assets in Zombie Arena

The game ships with a fixed set of **graphics/** files. Each one is a separate `Texture` loaded at startup.

#	File	Used for
1	`background_sheet.png`	Sprite sheet (mud-1, grass, mud-2, wall)
2	`player.png`	Player sprite
3	`bloater.png`	Bloater zombie (slow, tough)
4	`chaser.png`	Chaser zombie (fast)
5	`crawler.png`	Crawler zombie (average)
6	`bullet.png`	Bullet projectile
7	`crosshair.png`	Mouse crosshair / aim cursor
8	`ammo_icon.png`	HUD ammo pickup icon
9	`health_pickup.png`	Health pickup on the arena
10	`ammo_pickup.png`	Ammo pickup on the arena
11	`blood.png`	Blood splatter effect

Inside `background_sheet.png`:

The sprite sheet packs **4 distinct images** (tiles), each 50×50 px, stacked vertically: **mud-1** (0,0 → 50,50), **grass** (0,50 → 50,100), **mud-2** (0,100 → 50,150), and **wall** (0,150 → 50,200).

Totals: ~11 graphical asset files in `graphics/`; `background_sheet.png` contains **4** tile images.

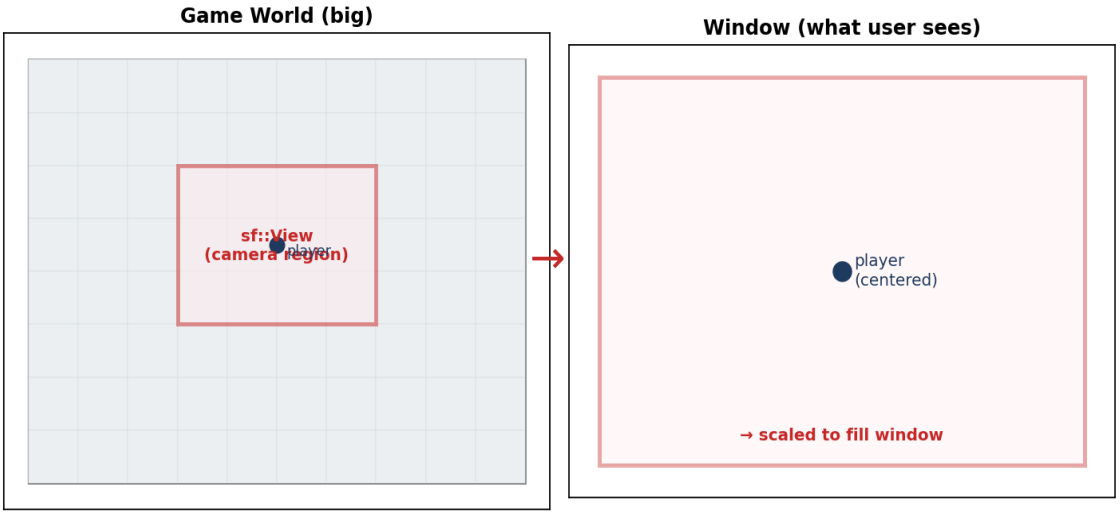
Q2. Sound files (.wav) and their triggers

All sounds live in `sound/` and are loaded into `SoundBuffer` objects, then played via `Sound`.

File	Played when…
`hit.wav`	A bullet hits a zombie
`splat.wav`	A zombie dies / blood splatter appears
`shoot.wav`	The player fires the gun
`reload.wav`	Reload completes successfully
`reload_failed.wav`	Reload attempted with 0 spare ammo
`powerup.wav`	Player levels up between waves
`pickup.wav`	Player walks over an ammo pickup

Q3. Two ways to create an sf::View

An `sf::View` is just a rectangle: it tells SFML which part of the world to draw and how big to draw it on screen. You can build one either from an explicit `FloatRect` or from a centre point + size.



Code:

```
#include <SFML/Graphics.hpp>
using namespace sf;

// ---- Method 1: View from a rectangle (left, top, width, height) ----
View view1(FloatRect(0.f, 0.f, 1920.f, 1080.f));

// ---- Method 2: View from centre + size ----
View view2(Vector2f(960.f, 540.f),     // centre point of the view
           Vector2f(1920.f, 1080.f));  // size (width, height)

// Both views above show the same region; just different ways to specify it.
```

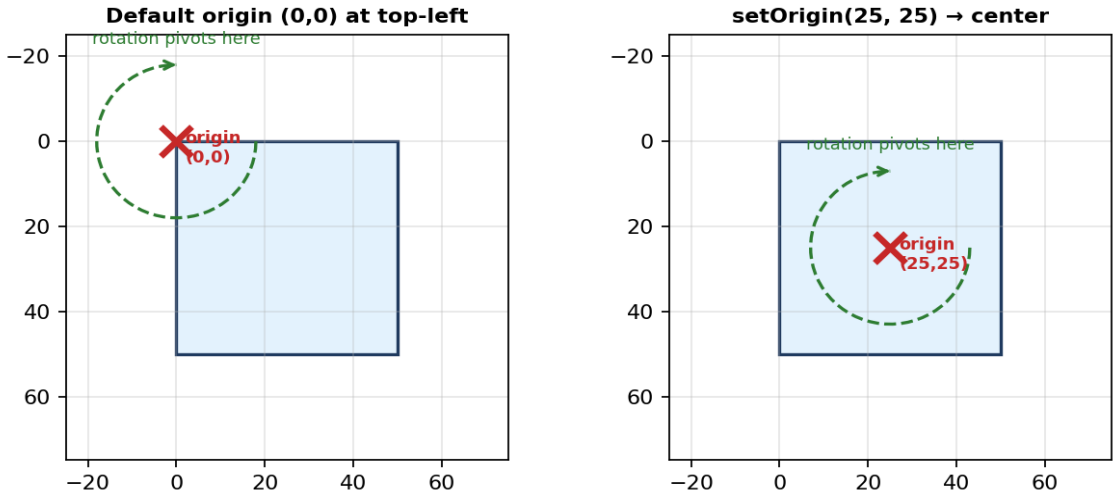
Q4. Apply a view to the window and draw with it

`setView` makes the view the *current* camera. Everything drawn afterwards is transformed by that view. To switch back to screen-space (e.g. for HUD), call `setView(window.getDefaultView())`.

```
// Set the view as the active camera
window.setView(mainView);

// Now draw all world-space objects through this view
window.clear();
window.draw(background, &textureBackground);
window.draw(playerSprite);
window.draw(zombieSprite);

// (optional) switch to default view for HUD elements
window.setView(window.getDefaultView());
window.draw(hudText);

window.display();
```

Q5. Fill in the missing names in the Player class skeleton

The image is 50×50 px, so the centre is (25, 25). `setOrigin(25, 25)` makes the sprite's pivot its own centre — vital for clean rotation.



Completed code:

```
class Player{
private:
    Texture m_Texture;     // texture data loaded from PNG
    Sprite  m_Sprite;      // drawable sprite that uses m_Texture
public:
    Player();
};

Player::Player(){
    m_Texture.loadFromFile("player.png");   // ?.?("player.png")
    m_Sprite.setTexture(m_Texture);         // ?.setTexture(?)
    m_Sprite.setOrigin(25, 25);             // image is 50x50 → centre = 25,25
    m_Sprite.setPosition(120, 234);         // ?.?(120,234)
}
```

Blank position	What goes there	Why
`Texture ?`	`m_Texture`	Member variable name
`Sprite ?`	`m_Sprite`	Member variable name
`?.?("player.png")`	`m_Texture.loadFromFile`	Load PNG into texture
`?.setTexture(?)`	`m_Sprite.setTexture(m_Texture)`	Attach texture to sprite
`?.setOrigin(?,?)`	`m_Sprite.setOrigin(25, 25)`	Centre of a 50×50 image
`?.?(120,234)`	`m_Sprite.setPosition(120, 234)`	Place sprite in world

Q6. Full Player class — draw at the centre of the view

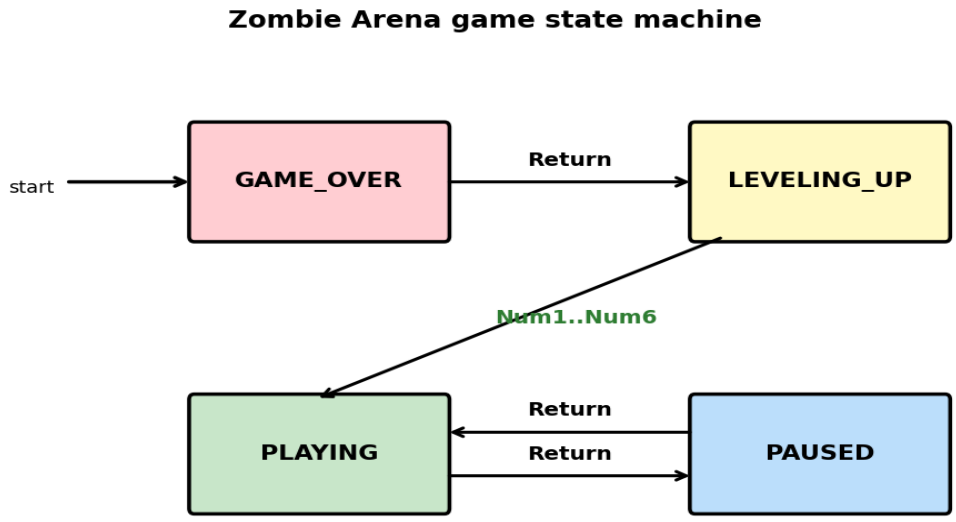
A clean class has the absolute minimum public surface: a constructor, a spawn that knows the arena and resolution, a way for the game to read back position and sprite, and an update each frame. Everything else stays private.

```
// ============ Player.h ============
#pragma once
#include <SFML/Graphics.hpp>
using namespace sf;

class Player{
private:
    // --- private data ---
    const float START_SPEED      = 200;
    const float START_HEALTH     = 100;

    Vector2f m_Position;         // current world position
    Sprite   m_Sprite;
    Texture  m_Texture;

    Vector2f m_Resolution;       // screen resolution
    IntRect  m_Arena;            // arena bounds in pixels
    int      m_TileSize;

    float    m_Speed   = START_SPEED;
    float    m_Health  = START_HEALTH;
    float    m_MaxHealth = START_HEALTH;

    // movement flags
    bool m_UpPressed, m_DownPressed, m_LeftPressed, m_RightPressed;

public:
    // --- public interface ---
    Player();

    void spawn(IntRect arena, Vector2f resolution, int tileSize);
    void update(float elapsedTime, Vector2f mousePosition);

    // setters for movement input
    void moveLeft();   void moveRight();
    void moveUp();     void moveDown();
    void stopLeft();   void stopRight();
    void stopUp();     void stopDown();

    // getters
    Vector2f getCenter()   const { return m_Position; }
    Sprite   getSprite()   const { return m_Sprite;   }
    float    getRotation() const { return m_Sprite.getRotation(); }
    FloatRect getPosition()const { return m_Sprite.getGlobalBounds(); }
};

// ============ Player.cpp ============
Player::Player(){
    m_Texture.loadFromFile("graphics/player.png");
    m_Sprite.setTexture(m_Texture);
    // origin at the centre of a 50x50 sprite
    m_Sprite.setOrigin(25, 25);
}

void Player::spawn(IntRect arena, Vector2f resolution, int tileSize){
    // spawn at the centre of the arena → which is the centre of the view
    m_Position.x = arena.width  / 2.0f;
    m_Position.y = arena.height / 2.0f;
```

```
    m_Arena      = arena;
    m_Resolution = resolution;
    m_TileSize   = tileSize;

    m_Sprite.setPosition(m_Position);
}
```

Why these members are private: outside code should never directly poke `m_Position` or `m_Health`. They go through **spawn / update / movement** methods so invariants (e.g. "player can't leave the arena") are always enforced inside the class.

Q7. Event polling + state machine



Code:

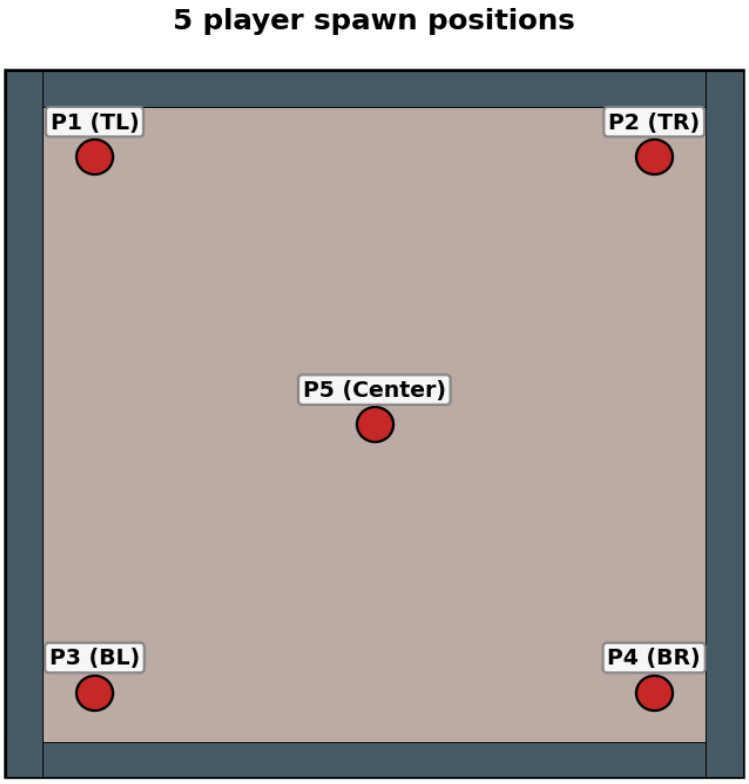
```
#include <SFML/Graphics.hpp>
#include <iostream>
using namespace sf;
using namespace std;

enum class State { PAUSED, LEVELING_UP, GAME_OVER, PLAYING };

int main(){
    RenderWindow window(VideoMode(1920, 1080), "Zombie Arena");
    State state = State::GAME_OVER;     // game starts on the title screen

    while (window.isOpen()) {

        // ----------- 1) POLL EVENTS -----------
        Event event;                    // instantiate event object
        while (window.pollEvent(event)) {

            if (event.type == Event::Closed)
                window.close();

            if (event.type == Event::KeyPressed) {

                // Return key: state changes
                if (event.key.code == Keyboard::Return) {

                    if (state == State::PLAYING) {
                        state = State::PAUSED;
                        cout << "State change: PLAYING -> PAUSED" << endl;
                    }
                    else if (state == State::PAUSED) {
                        state = State::PLAYING;
                        cout << "State change: PAUSED -> PLAYING" << endl;
                    }
                    else if (state == State::GAME_OVER) {
                        state = State::LEVELING_UP;
                        cout << "State change: GAME_OVER -> LEVELING_UP"
                             << endl;
                    }
                }

                // Quit on Escape
```

```
                if (event.key.code == Keyboard::Escape)
                    window.close();
            }
        }

        // ----------- 2) LEVELING_UP screen waits for Num1..Num6 -----------
        if (state == State::LEVELING_UP) {
            if (Keyboard::isKeyPressed(Keyboard::Num1)) {
                state = State::PLAYING;
                cout << "Upgrade #1 chosen -> PLAYING" << endl;
            }
            // (Num2..Num6 similarly upgrade different stats)
        }

        // ----------- 3) UPDATE & DRAW per state -----------
        window.clear();
        // ... per-state drawing ...
        window.display();
    }
    return 0;
}
```

Why `pollEvent` **in a loop?** Several events can queue up between frames (key press + mouse move + resize). Looping until the queue is empty processes them all in one frame, so input never feels laggy.

Q8. `spawn(...)` for 5 players

The original spawn always centres the player. To put a player anywhere, let spawn take explicit coordinates. Then create 5 Player objects and call spawn on each with a different position.



Modified spawn member function:

```
// Inside class Player (public):
void spawn(IntRect arena, Vector2f resolution, int tileSize,
           float startX, float startY);

// Implementation:
void Player::spawn(IntRect arena, Vector2f resolution, int tileSize,
                   float startX, float startY){
    m_Position.x = startX;
    m_Position.y = startY;

    m_Arena      = arena;
    m_Resolution = resolution;
    m_TileSize   = tileSize;

    m_Sprite.setPosition(m_Position);
}
```

Spawning 5 players + drawing:

```
IntRect arena(0, 0, 500, 500);
Vector2f resolution(1920, 1080);
int tileSize = 50;

// Create 5 player objects
Player player1, player2, player3, player4, player5;

// Spawn each at a different position
player1.spawn(arena, resolution, tileSize,  60,  60);    // top-left
player2.spawn(arena, resolution, tileSize, 440,  60);    // top-right
player3.spawn(arena, resolution, tileSize,  60, 440);    // bottom-left
player4.spawn(arena, resolution, tileSize, 440, 440);    // bottom-right
```

```
player5.spawn(arena, resolution, tileSize, 250, 250);    // centre

// Draw them all
window.draw(player1.getSprite());
window.draw(player2.getSprite());
window.draw(player3.getSprite());
window.draw(player4.getSprite());
window.draw(player5.getSprite());
```

Q9. Zombie class — spawn(startX, startY, type, seed)

Three zombie types exist: **BLOATER** (slow, lots of health), **CHASER** (fast, low health), **CRAWLER** (medium). The `type` parameter picks the texture and stats; `seed` seeds the RNG so each zombie's random properties are reproducible.

```
// ============ Zombie.h ============
#pragma once
#include <SFML/Graphics.hpp>
using namespace sf;

class Zombie{
private:
    // zombie type constants
    const int BLOATER  = 0;
    const int CHASER   = 1;
    const int CRAWLER  = 2;

    // per-type stats
    const float BLOATER_SPEED  = 40;
    const float CHASER_SPEED   = 75;
    const float CRAWLER_SPEED  = 50;
    const float BLOATER_HEALTH = 5;
    const float CHASER_HEALTH  = 1;
    const float CRAWLER_HEALTH = 3;

    float    m_Speed;
    float    m_Health;
    Vector2f m_Position;
    Sprite   m_Sprite;
    Texture  m_Texture;
    bool     m_Alive;
public:
    void spawn(float startX, float startY, int type, int seed);
    bool hit();
    bool isAlive() const { return m_Alive; }
    void update(float elapsedTime, Vector2f playerLocation);
    FloatRect getPosition() const { return m_Sprite.getGlobalBounds(); }
    Sprite getSprite()      const { return m_Sprite; }
};

// ============ Zombie.cpp ============
void Zombie::spawn(float startX, float startY, int type, int seed){
    switch (type) {
        case 0:      // BLOATER
            m_Texture.loadFromFile("graphics/bloater.png");
            m_Speed  = BLOATER_SPEED;
            m_Health = BLOATER_HEALTH;
            break;
        case 1:      // CHASER
            m_Texture.loadFromFile("graphics/chaser.png");
            m_Speed  = CHASER_SPEED;
            m_Health = CHASER_HEALTH;
            break;
        case 2:      // CRAWLER
            m_Texture.loadFromFile("graphics/crawler.png");
            m_Speed  = CRAWLER_SPEED;
            m_Health = CRAWLER_HEALTH;
            break;
    }
    // small per-zombie speed variation
    srand((int)time(0) * seed);
    float modifier = (rand() % 20) - 10;       // -10 .. +9
    m_Speed += modifier;
```

```
    m_Sprite.setTexture(m_Texture);
    m_Sprite.setOrigin(Vector2f(25, 25));
    m_Position.x = startX;
    m_Position.y = startY;
    m_Sprite.setPosition(m_Position);
    m_Alive = true;
}
```

THREE function-call statements with 3 independent zombies:

```
Zombie z1, z2, z3;

z1.spawn(100.0f, 100.0f, 0, 1);   // BLOATER at top
z2.spawn(500.0f, 250.0f, 1, 2);   // CHASER on the right
z3.spawn(300.0f, 450.0f, 2, 3);   // CRAWLER at the bottom
```

Q10. Use an *array* of Zombie objects instead of 3 separate ones

Replace the three named variables with a fixed-size array. Loop to spawn and to draw. Number of zombies known at compile time → stack array is fine.

```
const int NUM_ZOMBIES = 3;
Zombie zombies[NUM_ZOMBIES];          // array of 3 Zombie objects

// Spawn each one with different parameters
zombies[0].spawn(100.0f, 100.0f, 0, 1);   // BLOATER
zombies[1].spawn(500.0f, 250.0f, 1, 2);   // CHASER
zombies[2].spawn(300.0f, 450.0f, 2, 3);   // CRAWLER

// Update + draw them all
for (int i = 0; i < NUM_ZOMBIES; i++) {
    if (zombies[i].isAlive()) {
        window.draw(zombies[i].getSprite());
    }
}
```

Q11. Dynamic allocation with `new`

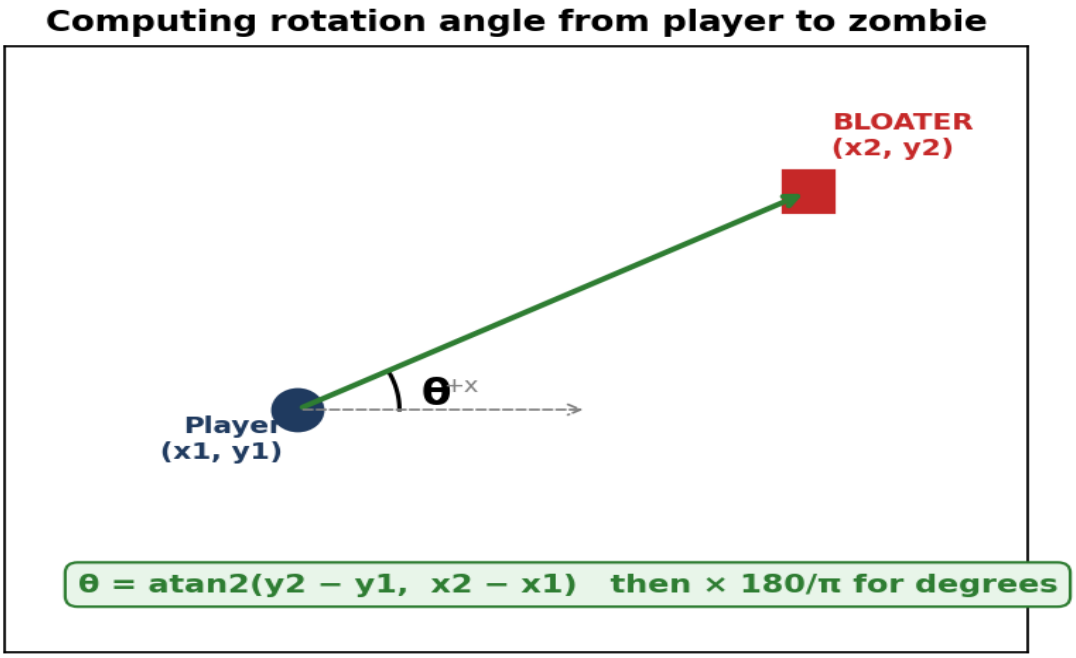
When the number of zombies depends on the wave / level (decided at runtime), allocate the array on the heap. Each wave can have a different size. Remember to `delete[]` when done to avoid memory leaks.

```
int numZombies = 3;     // could be 10, 100, whatever the wave needs

// Dynamic array of Zombie objects on the heap
Zombie* zombies = new Zombie[numZombies];

zombies[0].spawn(100.0f, 100.0f, 0, 1);   // BLOATER
zombies[1].spawn(500.0f, 250.0f, 1, 2);   // CHASER
zombies[2].spawn(300.0f, 450.0f, 2, 3);   // CRAWLER

// Draw loop
for (int i = 0; i < numZombies; i++) {
    if (zombies[i].isAlive()) {
        window.draw(zombies[i].getSprite());
    }
}

// IMPORTANT: free the heap memory before exiting
delete[] zombies;
zombies = nullptr;
```

Array vs new[] vs std::vector: static array → fixed at compile time, simplest. `new Zombie[N]` → size known at runtime, must manually `delete[]`. `std::vector<Zombie>` → resizes on the fly and cleans itself up; this is what the real Zombie Arena codebase uses.

Q12. Compute and apply rotation angle from player to BLOATER

The bloater should face the player. We use `atan2(dy, dx)` to get the angle in radians, convert to degrees (SFML wants degrees), then `setRotation` on the sprite.




```
#include <cmath>
#define PI 3.14159f

// (x1, y1) = player; (x2, y2) = BLOATER
float dx = x2 - x1;
float dy = y2 - y1;

// atan2 returns radians; convert to degrees for SFML
float angle = atan2f(dy, dx) * 180.0f / PI;

// Apply to the bloater sprite
m_Sprite.setRotation(angle);
```

Why `atan2` and not `atan(dy/dx)`? atan2 looks at the signs of both dy and dx, so it returns the correct angle in *all four quadrants*. Plain atan only covers two quadrants and divides by zero when dx = 0.

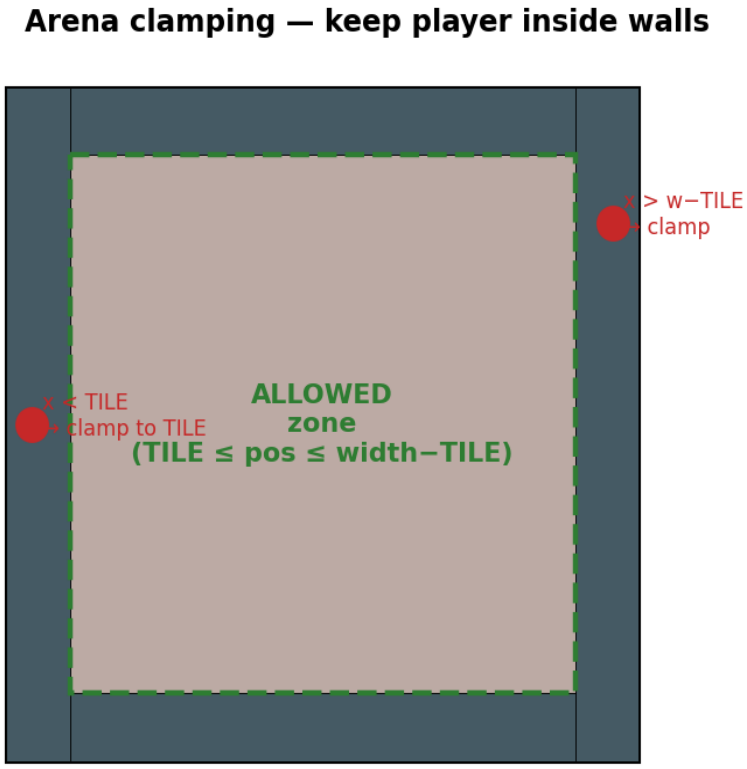
Q13. Zombie sprite is to the LEFT of the player — update position

If the zombie is left of the player, then `m_Position.x < playerLocation.x`. To make the zombie chase the player, we move it slightly to the right each frame (positive x direction), and match the player's y.

```
// zombie is to the LEFT of the player → move it RIGHT toward the player
// (m_Speed is units per second, dt is the frame's elapsed time in seconds)

m_Position.x += m_Speed * dt;          // step toward player on x-axis
m_Position.y  = playerLocation.y;      // align vertically (simple chase)

// Push the new position into the sprite
m_Sprite.setPosition(m_Position);

// ---- Alternative one-liner used in the textbook code ----
// m_Position.x = m_Position.x + (m_Speed * dt);
```

Q14. Keep player inside the arena (clamp against walls)

After moving, check each axis. If the player has gone past the wall thickness on any side, snap them back to the boundary. The wall is `m_TileSize` (50 px) thick on every edge.




```
// Left edge
if (m_Position.x < m_Arena.left + m_TileSize) {
    m_Position.x = m_Arena.left + m_TileSize;
}

// Right edge
if (m_Position.x > m_Arena.width - m_TileSize) {
    m_Position.x = m_Arena.width - m_TileSize;
}

// Top edge
if (m_Position.y < m_Arena.top + m_TileSize) {
    m_Position.y = m_Arena.top + m_TileSize;
}

// Bottom edge
if (m_Position.y > m_Arena.height - m_TileSize) {
    m_Position.y = m_Arena.height - m_TileSize;
}

// Apply the (possibly clamped) position
m_Sprite.setPosition(m_Position);
```

Note: we test against `arena.width / height` directly because in the Zombie Arena codebase `m_Arena.left` and `m_Arena.top` are 0, so width/height behave as the right/bottom edges.

Q15. Random integer between 80 and 100

Classic `rand() % range` trick. Range size is **100** − **80 + 1 = 21**, then shift up by 80.

```
#include <cstdlib>
#include <ctime>
```

```
// Seed once at program start (otherwise you get the same sequence each run)
srand((unsigned)time(0));

// Generate a random number in [80, 100]
int randomNumber = (rand() % 21) + 80;
// rand() % 21  → 0..20
// + 80         → 80..100
```

General formula: for any range [a, b], use
`(rand() % (b - a + 1)) + a`

Quick reference — cheat sheet for the exam

sf::View patterns

```
View v1(FloatRect(0, 0, 1920, 1080));            // from rect
View v2(Vector2f(960, 540), Vector2f(1920, 1080)); // from centre+size

window.setView(v1);          // apply as current camera
window.draw(...);            // drawn through v1
window.setView(window.getDefaultView());   // back to screen-space
```

Class skeleton (Player or Zombie)

```
class X {
private:
    Texture  m_Texture;
    Sprite   m_Sprite;
    Vector2f m_Position;
    float    m_Speed, m_Health;
public:
    void spawn(...);
    void update(float dt, Vector2f playerPos);
    Sprite getSprite() const { return m_Sprite; }
    Vector2f getCenter() const { return m_Position; }
};
```

Sprite origin = setOrigin(w/2, h/2)

Always set the origin to the centre of the sprite for clean rotation. For a 50×50 image, `setOrigin(25, 25)`.

Rotation toward a target

```
float angle = atan2f(target.y - me.y, target.x - me.x) * 180.0f / 3.14159f;
m_Sprite.setRotation(angle);
```

Arena clamp (4 conditions)

```
if (pos.x < arena.left   + TILE) pos.x = arena.left   + TILE;
if (pos.x > arena.width  - TILE) pos.x = arena.width  - TILE;
if (pos.y < arena.top    + TILE) pos.y = arena.top    + TILE;
if (pos.y > arena.height - TILE) pos.y = arena.height - TILE;
```

Random integer in [a, b]

```
int n = (rand() % (b - a + 1)) + a;
```

> **Viva tip:** the whole point of `sf::View` is that you write all your game logic in *world coordinates* and let SFML do the camera math. Move the view to follow the player, and the world appears to scroll — without changing any `setPosition` calls.